



UNIVERSIDAD DE EL SALVADOR
Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura
Ingeniería en Desarrollo de Software



Asignatura:

Desarrollo de Aplicaciones Web

Ciclo V / Tercer año

Proyecto - Healify

Laboratorio 2: Backend Funcional

Coordinador de Cátedra:

Ing. Alexis Guardado

Tutor GT01:

Ing. Andrea Victoria Castro Jiménez

Integrantes del Grupo 07:

MO21016	Jason Alexander Molina Ortiz
RM24001	Cindy Ariana Reyes Molina
GP24005	Rodrigo Ernesto Garcia Portillo
NR24002	Marlon Alexis Núñez Ramos
CA18052	Hugo Fernando Canizales Andrade

Lugar y fecha de entrega:

Ciudad Universitaria, Domingo 12 de abril de 2026

<https://github.com/MO21016/DAW-Sistema-Gestion-Citas-Medicas.git>

Tabla de Contenido

Resumen	3
1. Diseño de Base de Datos.....	4
1.1 Diagrama de Entidad-Relación	4
1.2 Script de Creación SQL.....	4
1.2.1 Creación de la base de datos	4
1.2.2 Inserción de datos de prueba	6
2. Arquitectura Backend.....	9
Implementación de Arquitectura en Capas.....	10
Uso de DTOs y Mapeo de Datos	10
Ejemplo de Lógica de Negocio y Mapeo	11
Validaciones y Reglas de Negocio.....	11
Flujo de Datos en el Sistema	12
3. Evidencia de funcionalidad	12
3.1 Vista general de la documentación.....	13
3.2 Prueba de operaciones.....	14
3.2.1 POST	14
3.2.2 GET	16
3.2.3. PUT	17
3.2.4. DELETE	18
4. Validación de base de datos	19

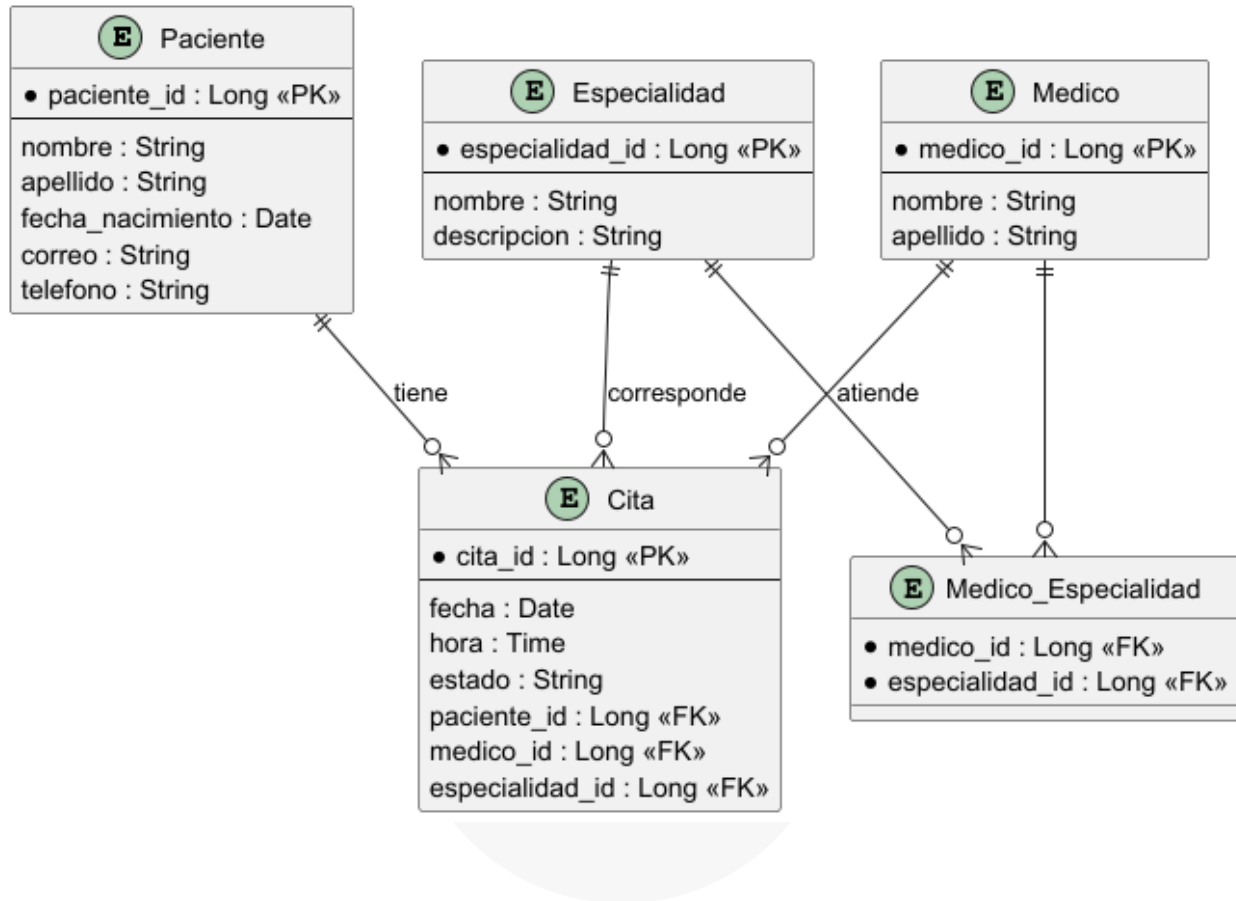
Resumen

Healify es un sistema de gestión de citas médicas diseñado bajo una arquitectura en capas Cliente-Servidor. Este sistema permite a clínicas y centros de salud administrar de forma eficiente la programación de citas, mantener un historial organizado de cada paciente y gestionar el listado de médicos junto con sus especialidades. Además, ayuda a evitar conflictos de horario mediante el control de disponibilidad de citas.

Todo esto se logra gracias a una base de datos estructurada que almacena la información de manera segura y ordenada, permitiendo un acceso rápido y confiable. El sistema optimiza los procesos administrativos, reduce errores humanos y mejora la calidad del servicio brindado a los pacientes. Asimismo, facilita la toma de decisiones mediante información centralizada y actualizada en tiempo real.

1. Diseño de Base de Datos

1.1 Diagrama de Entidad-Relación



1.2 Script de Creación SQL

1.2.1 Creación de la base de datos

```
-- =====
-- Sistema de Gestión de Citas Médicas
-- =====

CREATE TABLE IF NOT EXISTS medico (
  id_medico BIGSERIAL PRIMARY KEY,
  nombre_medico VARCHAR(100) NOT NULL,
  apellido_medico VARCHAR(100) NOT NULL,
  telefono_medico VARCHAR(15) NOT NULL,
  correo_medico VARCHAR(100) NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



```

CREATE INDEX IF NOT EXISTS idx_correo_medico ON medico(correo_medico);
CREATE INDEX IF NOT EXISTS idx_nombre_medico ON medico(nombre_medico,
apellido_medico);

-- =====
CREATE TABLE IF NOT EXISTS especialidad (
    id_especialidad BIGSERIAL PRIMARY KEY,
    nombre_especialidad VARCHAR(100) NOT NULL UNIQUE,
    descripcion VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_nombre_especialidad ON
especialidad(nombre_especialidad);

-- =====
CREATE TABLE IF NOT EXISTS medico_especialidad (
    id_medico_especialidad BIGSERIAL PRIMARY KEY,
    id_medico BIGINT NOT NULL,
    id_especialidad BIGINT NOT NULL,
    fecha_asignacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_medico) REFERENCES medico(id_medico) ON DELETE CASCADE,
    FOREIGN KEY (id_especialidad) REFERENCES especialidad(id_especialidad) ON
DELETE CASCADE,
    UNIQUE (id_medico, id_especialidad)
);

CREATE INDEX IF NOT EXISTS idx_medico ON medico_especialidad(id_medico);
CREATE INDEX IF NOT EXISTS idx_especialidad ON
medico_especialidad(id_especialidad);

-- =====
CREATE TABLE IF NOT EXISTS paciente (
    id_paciente BIGSERIAL PRIMARY KEY,
    nombre_paciente VARCHAR(100) NOT NULL,
    apellido_paciente VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    telefono_paciente VARCHAR(15) NOT NULL,
    correo_paciente VARCHAR(100) NOT NULL UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_correo_paciente ON paciente(correo_paciente);
CREATE INDEX IF NOT EXISTS idx_nombre_paciente ON paciente(nombre_paciente,
apellido_paciente);

```



```

-- =====
CREATE TYPE estado_cita_enum AS ENUM ('PENDIENTE', 'CONFIRMADA', 'CANCELADA',
'COMPLETADA');

CREATE TABLE IF NOT EXISTS cita (
    id_cita BIGSERIAL PRIMARY KEY,
    fecha_cita DATE NOT NULL,
    hora_cita TIME NOT NULL,
    motivo_cita VARCHAR(255) NOT NULL,
    estado_cita estado_cita_enum NOT NULL DEFAULT 'PENDIENTE',
    id_paciente BIGINT NOT NULL,
    id_medico BIGINT NOT NULL,
    id_especialidad BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_paciente) REFERENCES paciente(id_paciente) ON DELETE
RESTRICT,
    FOREIGN KEY (id_medico) REFERENCES medico(id_medico) ON DELETE RESTRICT,
    FOREIGN KEY (id_especialidad) REFERENCES especialidad(id_especialidad) ON
DELETE RESTRICT
);

CREATE INDEX IF NOT EXISTS idx_fecha_cita ON cita(fecha_cita);
CREATE INDEX IF NOT EXISTS idx_estado_cita ON cita(estado_cita);
CREATE INDEX IF NOT EXISTS idx_medico_fecha ON cita(id_medico, fecha_cita,
hora_cita);
CREATE INDEX IF NOT EXISTS idx_paciente ON cita(id_paciente);

```

1.2.2 Inserción de datos de prueba

```

-- =====
-- ESPECIALIDADES (10 especialidades comunes)
-- =====
INSERT INTO especialidad (nombre_especialidad, descripcion) VALUES
('Cardiología', 'Diagnóstico y tratamiento de enfermedades del corazón'),
('Pediatría', 'Atención médica de bebés, niños y adolescentes'),
('Dermatología', 'Diagnóstico y tratamiento de enfermedades de la piel'),
('Traumatología', 'Tratamiento de lesiones del sistema musculoesquelético'),
('Ginecología', 'Atención de la salud de la mujer'),
('Oftalmología', 'Diagnóstico y tratamiento de enfermedades de los ojos'),
('Neurología', 'Tratamiento de enfermedades del sistema nervioso'),
('Psiquiatría', 'Diagnóstico y tratamiento de trastornos mentales'),
('Medicina General', 'Atención médica integral y preventiva'),
('Odontología', 'Diagnóstico y tratamiento de enfermedades bucales');

-- =====
-- MÉDICOS (12 médicos)
-- =====
INSERT INTO medico (nombre_medico, apellido_medico, telefono_medico,
correo_medico) VALUES
('Carlos', 'Rodríguez', '77001234', 'carlos.rodriguez@hospital.com'),
('María', 'González', '77005678', 'maria.gonzalez@hospital.com'),

```



```

('José', 'Martínez', '77009012', 'jose.martinez@hospital.com'),
('Ana', 'López', '77003456', 'ana.lopez@hospital.com'),
('Roberto', 'Hernández', '77007890', 'roberto.hernandez@hospital.com'),
('Laura', 'García', '77002345', 'laura.garcia@hospital.com'),
('Pedro', 'Ramírez', '77006789', 'pedro.ramirez@hospital.com'),
('Carmen', 'Torres', '77000123', 'carmen.torres@hospital.com'),
('Luis', 'Flores', '77004567', 'luis.flores@hospital.com'),
('Patricia', 'Morales', '77008901', 'patricia.morales@hospital.com'),
('Miguel', 'Castillo', '77001122', 'miguel.castillo@hospital.com'),
('Sofía', 'Vásquez', '77005566', 'sofia.vasquez@hospital.com');

-- =====
-- ASIGNACIÓN DE ESPECIALIDADES A MÉDICOS
-- =====
INSERT INTO medico_especialidad (id_medico, id_especialidad) VALUES
(1, 1), (1, 9),
(2, 2),
(3, 3),
(4, 4), (4, 9),
(5, 5),
(6, 6),
(7, 7), (7, 9),
(8, 8),
(9, 9),
(10, 10),
(11, 1),
(12, 2), (12, 9);

-- =====
-- PACIENTES (15 pacientes)
-- =====
INSERT INTO paciente (nombre_paciente, apellido_paciente, fecha_nacimiento,
telefono_paciente, correo_paciente) VALUES
('Juan', 'Pérez', '1985-03-15', '70001234', 'juan.perez@email.com'),
('María', 'Sánchez', '1990-07-22', '70005678', 'maria.sanchez@email.com'),
('Carlos', 'Mendoza', '1978-11-30', '70009012', 'carlos.mendoza@email.com'),
('Ana', 'Reyes', '1995-01-10', '70003456', 'ana.reyes@email.com'),
('Roberto', 'Cruz', '1982-05-18', '70007890', 'roberto.cruz@email.com'),
('Laura', 'Díaz', '1988-09-25', '70002345', 'laura.diaz@email.com'),
('Pedro', 'Romero', '2010-02-14', '70006789', 'pedro.romero@email.com'),
('Carmen', 'Silva', '1975-12-05', '70000123', 'carmen.silva@email.com'),
('Luis', 'Ortiz', '1992-04-20', '70004567', 'luis.ortiz@email.com'),
('Patricia', 'Navarro', '1987-08-12', '70008901',
'patricia.navarro@email.com'),
('Miguel', 'Campos', '2015-06-30', '70001122', 'miguel.campos@email.com'),
('Sofía', 'Aguilar', '1993-10-08', '70005566', 'sofia.aguilar@email.com'),
('Diego', 'Mejía', '1980-03-27', '70009900', 'diego.mejia@email.com'),
('Gabriela', 'Ramos', '1998-07-15', '70003344', 'gabriela.ramos@email.com'),
('Fernando', 'Vargas', '1970-11-22', '70007788', 'fernando.vargas@email.com');

```



```
-- =====
-- CITAS (20 citas de ejemplo)
-- =====

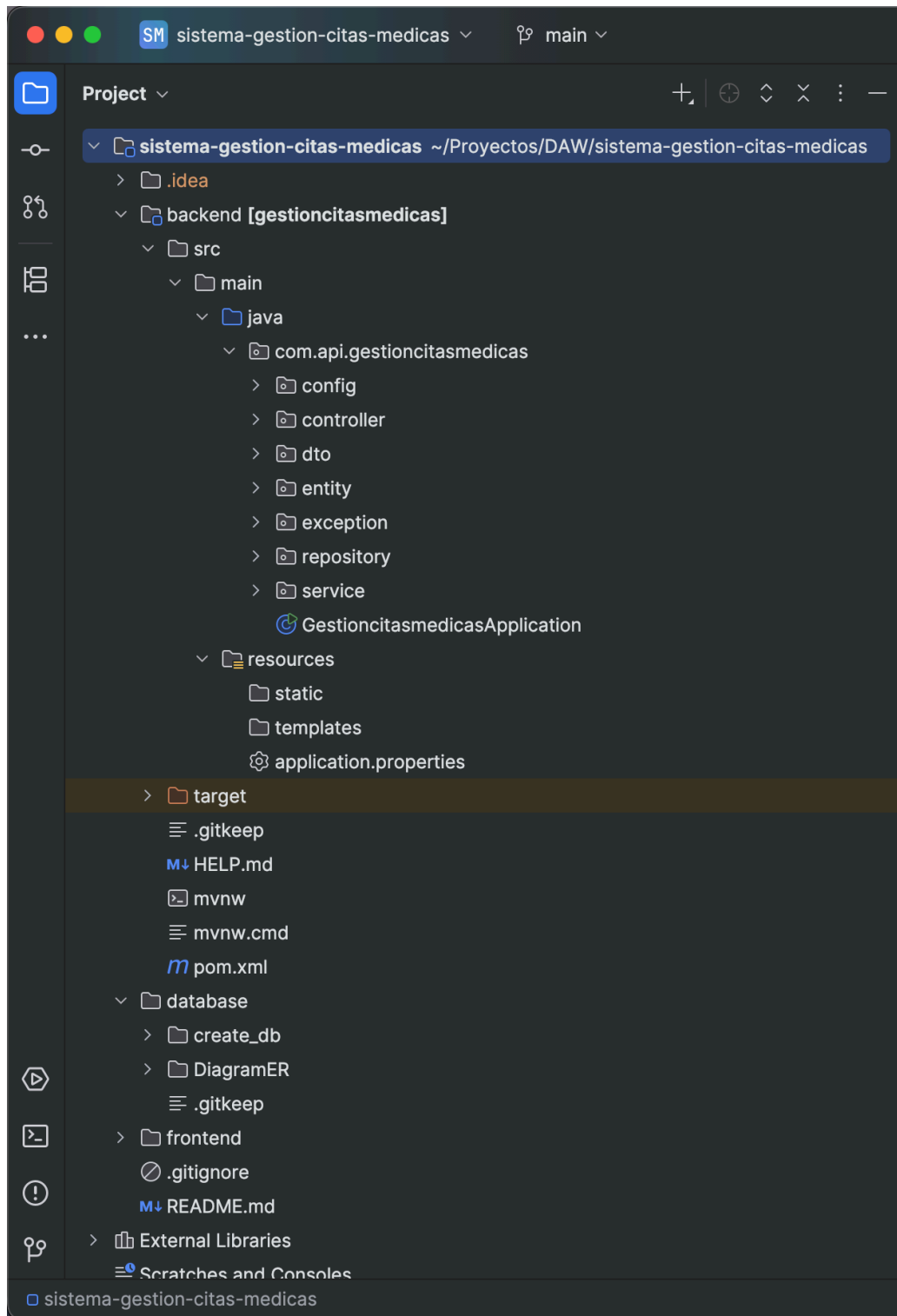
INSERT INTO cita (fecha_cita, hora_cita, motivo_cita, estado_cita,
id_paciente, id_medico, id_especialidad) VALUES
('2025-10-15', '08:00:00', 'Chequeo cardiológico de rutina',
'CONFIRMADA'::estado_cita_enum, 1, 1, 1),
('2025-10-15', '09:00:00', 'Control de crecimiento',
'CONFIRMADA'::estado_cita_enum, 7, 2, 2),
('2025-10-15', '10:00:00', 'Consulta por acné',
'CONFIRMADA'::estado_cita_enum, 4, 3, 3),
('2025-10-15', '14:00:00', 'Dolor de rodilla', 'CONFIRMADA'::estado_cita_enum,
3, 4, 4),
('2025-10-16', '08:00:00', 'Control prenatal', 'CONFIRMADA'::estado_cita_enum,
6, 5, 5),
('2025-10-17', '08:00:00', 'Revisión de vista', 'PENDIENTE'::estado_cita_enum,
8, 6, 6),
('2025-10-17', '09:00:00', 'Dolor de cabeza recurrente',
'PENDIENTE'::estado_cita_enum, 9, 7, 7),
('2025-10-17', '10:00:00', 'Consulta por ansiedad',
'PENDIENTE'::estado_cita_enum, 10, 8, 8),
('2025-10-17', '14:00:00', 'Chequeo general', 'PENDIENTE'::estado_cita_enum,
2, 9, 9),
('2025-10-18', '08:00:00', 'Limpieza dental', 'PENDIENTE'::estado_cita_enum,
11, 10, 10),
('2025-10-01', '08:00:00', 'Electrocardiograma',
'COMPLETADA'::estado_cita_enum, 5, 1, 1),
('2025-10-02', '09:00:00', 'Vacunación infantil',
'COMPLETADA'::estado_cita_enum, 11, 12, 2),
('2025-10-03', '10:00:00', 'Tratamiento dermatológico',
'COMPLETADA'::estado_cita_enum, 12, 3, 3),
('2025-10-04', '14:00:00', 'Fisioterapia', 'COMPLETADA'::estado_cita_enum, 13,
4, 4),
('2025-10-05', '08:00:00', 'Ecografía', 'COMPLETADA'::estado_cita_enum, 14, 5,
5),
('2025-10-20', '08:00:00', 'Consulta oftalmológica',
'CANCELADA'::estado_cita_enum, 15, 6, 6),
('2025-10-20', '09:00:00', 'Evaluación neurológica',
'CANCELADA'::estado_cita_enum, 1, 7, 7),
('2025-10-20', '10:00:00', 'Terapia psicológica',
'CANCELADA'::estado_cita_enum, 2, 8, 8),
('2025-10-21', '08:00:00', 'Consulta general', 'CANCELADA'::estado_cita_enum,
3, 9, 9),
('2025-10-21', '09:00:00', 'Extracción dental', 'CANCELADA'::estado_cita_enum,
4, 10, 10);

-- =====
-- Verificación
-- =====

SELECT 'Datos de prueba insertados exitosamente' AS estado;
SELECT COUNT(*) AS total_especialidades FROM especialidad;
SELECT COUNT(*) AS total_medicos FROM medico;
SELECT COUNT(*) AS total_pacientes FROM paciente;
SELECT COUNT(*) AS total_citas FROM cita;
```



2. Arquitectura Backend



Implementación de Arquitectura en Capas

El backend del sistema *Healify* fue desarrollado siguiendo una arquitectura en capas, permitiendo una clara separación de responsabilidades y facilitando el mantenimiento y escalabilidad del sistema

La estructura del proyecto se organiza en los siguientes paquetes:

- **controller:** Encargado de exponer los endpoints REST.
- **service:** Contiene la lógica de negocio del sistema.
- **repository:** Maneja el acceso a datos mediante interfaces que extienden JpaRepository.
- **entity:** Representa las entidades de la base de datos.
- **dto:** Define los objetos de transferencia de datos.
- **exception:** Manejo de errores personalizados.

Esta organización asegura el desacoplamiento entre capas y el cumplimiento del patrón Cliente-Servidor

Uso de DTOs y Mapeo de Datos

Para evitar exponer directamente las entidades de la base de datos, se implementó el uso de **DTOs (Data Transfer Objects)**, permitiendo un mejor control sobre la información enviada y recibida por la API

Para esta entrega inicial, se definieron tres tipos de DTOs para la entidad *Especialidad*:

- **EspecialidadDTO:** Utilizado para enviar datos al cliente.
- **CrearEspecialidadDTO:** Utilizado para la creación de nuevas especialidades.
- **ActualizarEspecialidadDTO:** Utilizado para la actualización de datos existentes.

Además, se implementa el proceso de conversión entre entidades y DTOs dentro de la capa de servicio, cumpliendo con el principio de desacoplamiento



Ejemplo de Lógica de Negocio y Mapeo

La lógica de negocio se implementa en la clase EspecialidadService, donde se gestionan las operaciones CRUD y se realiza el mapeo entre entidades y DTOs.

Para la conversión de una entidad a DTO, se utiliza el siguiente método:

```
private EspecialidadDTO convertirADTO(Especialidad especialidad) {  
    EspecialidadDTO dto = new EspecialidadDTO();  
    dto.setIdEspecialidad(especialidad.getIdEspecialidad());  
    dto.setNombreEspecialidad(especialidad.getNombreEspecialidad());  
    dto.setDescripcion(especialidad.getDescripcion());  
    return dto;  
}
```

Este método permite transformar los datos obtenidos desde la base de datos en un formato adecuado para ser enviado al cliente.

Validaciones y Reglas de Negocio

Dentro de la capa de servicio también se implementan validaciones importantes, tales como:

- Verificación de existencia antes de actualizar o eliminar registros.
- Validación de unicidad en el nombre de la especialidad.
- Control de campos opcionales en actualizaciones.

Esto garantiza la integridad de los datos y evita inconsistencias en el sistema.



Flujo de Datos en el Sistema

El flujo de datos en la aplicación sigue el siguiente proceso:

1. El cliente envía una solicitud HTTP al **Controller**.
2. El Controller recibe un DTO de entrada.
3. El **Service** procesa la lógica de negocio.
4. Se realiza el mapeo de DTO a Entidad para persistencia.
5. El **Repository** interactúa con la base de datos.
6. Se obtiene la entidad y se convierte nuevamente a DTO.
7. El DTO es retornado al cliente como respuesta.

3. Evidencia de funcionalidad

Para la documentación del backend se implementó **Swagger mediante OpenAPI**, lo que permite visualizar, probar y validar los endpoints de manera interactiva desde una interfaz web

Esta herramienta facilita la comprensión de la API, permitiendo a los desarrolladores y usuarios conocer los servicios disponibles, los parámetros requeridos y las respuestas generadas por el sistema

Se implementaron y documentaron los cuatro métodos principales del CRUD para la gestión de entidades:

- **GET:** Permite obtener información (listar o buscar registros).
- **POST:** Permite crear nuevos registros en el sistema.
- **PUT:** Permite actualizar información existente.
- **DELETE:** Permite eliminar registros.

Cada uno de estos endpoints fue documentado utilizando anotaciones como `@Operation` y `@Tag`, lo que permite describir su funcionalidad dentro de Swagger

3.1 Vista general de la documentación

 **Swagger**
Facilita SMARTBEAR

/v3/api-docs

Explore

API de Gestión de Citas Médicas

1.0.0 OAS 3.1

/v3/api-docs

Sistema para la administración de citas médicas.

Contact DAW-Grupo1

Servers

http://localhost:8080 - Generated server url

1. Health

Endpoints para verificar el estado del servidor.

GET

/api/info

Información del proyecto

▼

GET

/api/health

Verificar estado del servidor

▼

2. Especialidades

Operaciones para la gestión de especialidades médicas.

DELETE

/api/especialidades/{id}

Eliminar especialidad

▼

GET

/api/especialidades/{id}

Obtener especialidad por ID

▼

GET

/api/especialidades

Listar todas las especialidades

▼

POST

/api/especialidades

Crear nueva especialidad

▼

PUT

/api/especialidades/{id}

Actualizar especialidad

▼

Schemas

ActualizarEspecialidadDTO >

Expand all

object

EspecialidadDTO >

Expand all

object

CrearEspecialidadDTO >

Expand all

object

3.2 Prueba de operaciones

3.2.1 POST

Ejemplo 1

POST /api/especialidades Crear nueva especialidad

Registra una nueva especialidad médica en el sistema.

Parameters Cancel

No parameters

Request body required application/json

```
{
  "nombreEspecialidad": "string",
  "descripcion": "string"
}
```

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  http://localhost:8080/api/especialidades \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombreEspecialidad": "string",
    "descripcion": "string"
  }'
```

Request URL

http://localhost:8080/api/especialidades

Server response

Code	Details
201	Response body <pre>{ "idEspecialidad": 11, "nombreEspecialidad": "string", "descripcion": "string", "cantidadMedicos": null }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Sun, 12 Apr 2020 13:12:32 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

Media type */*

Content Accept header

Example Value | **Schema**

```
{
  "idEspecialidad": 5007199254740093,
  "nombreEspecialidad": "string",
  "descripcion": "string",
  "cantidadMedicos": 1073741824
}
```

Ejemplo 2

```
{
  "nombreEspecialidad": "Medicina interna",
  "descripcion": "Diagnostico y tratamiento de enfermedades de adulto"
}
```

Execute

Clear

Responses

```
curl -X 'POST' \
  'http://localhost:8080/api/especialidades' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombreEspecialidad": "Medicina interna",
    "descripcion": "Diagnostico y tratamiento de enfermedades de adulto"
  }'

{"nombreEspecialidad": "Geriatrica",
 "descripcion": "Cuidado de la salud de los adultos"
}
```

Request URL

http://localhost:8080/api/especialidades

Server response

Code Details

1

documented

Response body

```
{
  "idEspecialidad": 13,
  "nombreEspecialidad": "Medicina interna",
  "descripcion": "Diagnostico y tratamiento de enfermedades de adulto",
  "cantidadMedicos": null
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Sun, 12 Apr 2026 13:36:10 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code Description

0

OK

Media type

/

Content & Content header



Ingeniería en
DESARROLLO DE SOFTWARE

3.2.2 GET

GET /api/especialidades Lista todas las especialidades

Retorna una lista con todas las especialidades médicas registradas en el sistema.

Parameters Cancel

No parameters

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/especialidades' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/especialidades
```

Server response

Code	Details
200	Response body<pre>{ "idEspecialidad": 1, "nombreEspecialidad": "Cardiología", "descripcion": "Diagnostico y tratamiento de enfermedades del corazón", "cantidadMedicos": null }, { "idEspecialidad": 2, "nombreEspecialidad": "Pediatría", "descripcion": "Atención médica de bebés, niños y adolescentes", "cantidadMedicos": null }, { "idEspecialidad": 3, "nombreEspecialidad": "Dermatología", "descripcion": "Diagnostico y tratamiento de enfermedades de la piel", "cantidadMedicos": null }, { "idEspecialidad": 4, "nombreEspecialidad": "Traumatología", "descripcion": "Tratamiento de lesiones del sistema musculoesquelético", "cantidadMedicos": null } }</pre>Download Response headers<pre>connection: keep-alive content-type: application/json date: Sun, 12 Apr 2025 13:16:18 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

Media type
application/json

Control Accept header

Example Value | **Schema**

```
{
  "idEspecialidad": 9087193254748993,
  "nombreEspecialidad": "string",
  "descripcion": "string",
  "cantidadMedicos": 1873741824
}
```



3.2.3. PUT

PUT

/api/especialidades/{id} Actualizar especialidad

Actualiza el nombre o descripción de una especialidad existente según su ID.

Parameters

Cancel

Reset

Name	Description
id * required integer(\$int64) (path)	12

Request body required

application/json

```
{  
  "nombreEspecialidad": "Oftalmología",  
  "descripcion": "Tratamiento, diagnóstico y prevención de enfermedades Oculares"  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \  
  'http://localhost:8080/api/especialidades/12' \  
  -H 'accept: */*' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "nombreEspecialidad": "Oftalmología",  
    "descripcion": "Tratamiento, diagnóstico y prevención de enfermedades Oculares"  
  }'
```

Request URL

```
http://localhost:8080/api/especialidades/12
```

Server response

Code	Details
200	Response body <pre>{ "idEspecialidad": 12, "nombreEspecialidad": "Oftalmología", "descripcion": "Tratamiento, diagnóstico y prevención de enfermedades Oculares", "cantidadMedicos": null }</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Sun, 12 Apr 2026 13:26:47 GMT keep-alive: +100000000</pre>



3.2.4. DELETE

DELETE /api/especialidades/{id} Eliminar especialidad

Elimina una especialidad del sistema según su ID.

Parameters Cancel

Name	Description
id * required integer(\$int64) (path)	11

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:8080/api/especialidades/11' \
-H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/especialidades/11
```

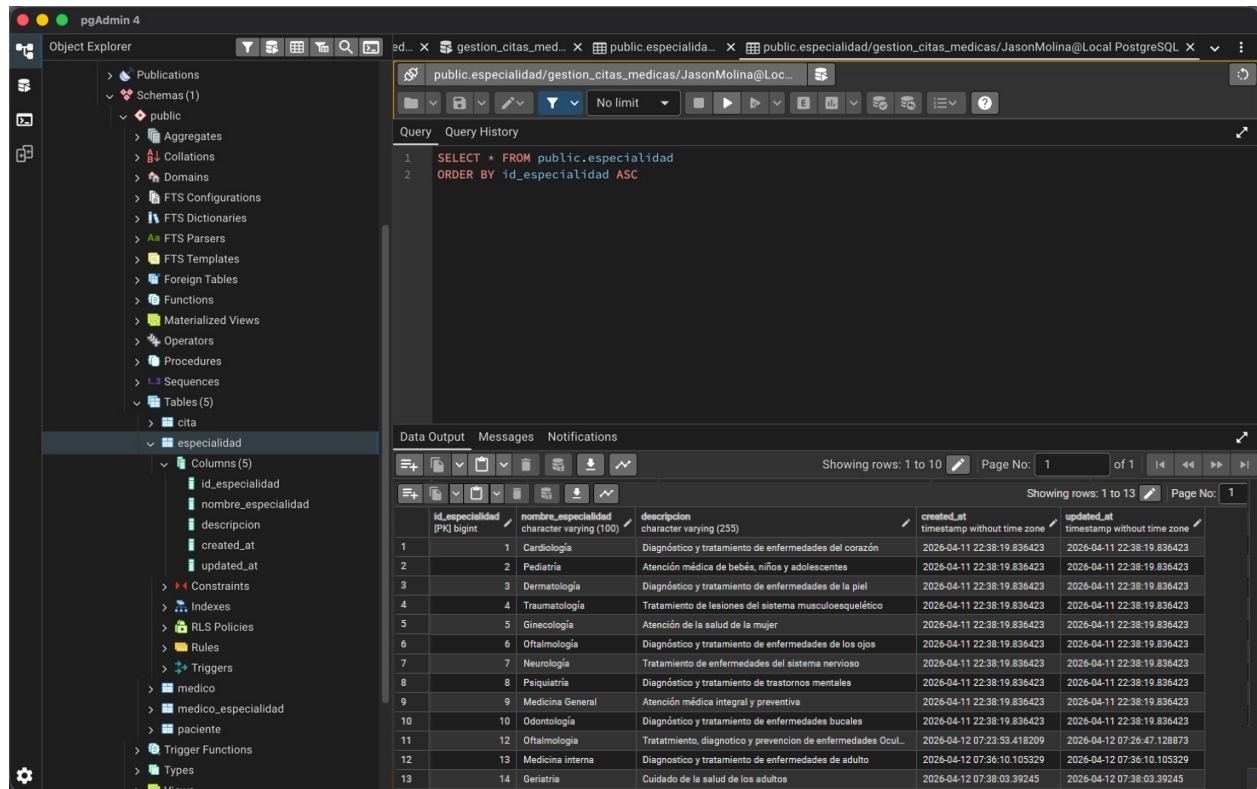
Server response

Code	Details
204 Undocumented	Response headers connection: keep-alive date: Sun, 12 Apr 2026 13:21:25 GMT keep-alive: timeout=60

Responses

Code	Description	Links
200	OK	No links

4. Validación de base de datos



The screenshot displays the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure, including the 'public' schema and the 'especialidad' table. The central pane shows a SQL query: `SELECT * FROM public.especialidad ORDER BY id_especialidad ASC`. The bottom pane displays the query results in a table format, showing 13 rows of data.

id_especialidad (PK) bigint	nombre_especialidad character varying (100)	descripcion character varying (255)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	Cardiología	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
2	2	Pediatría	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
3	3	Dermatología	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
4	4	Traumatología	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
5	5	Ginecología	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
6	6	Oftalmología	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
7	7	Neurología	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
8	8	Psiquiatría	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
9	9	Medicina General	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
10	10	Odontología	2026-04-11 22:38:19.836423	2026-04-11 22:38:19.836423
11	12	Oftalmología	2026-04-12 07:23:33.418209	2026-04-12 07:26:47.128873
12	13	Medicina interna	2026-04-12 07:36:10.105329	2026-04-12 07:36:10.105329
13	14	Geriatría	2026-04-12 07:38:03.39245	2026-04-12 07:38:03.39245